Preliminaries

Ricardo Andres Calvo Mendez

PUID: 36688820 · rcalvome@purdue.edu

ECE 69500 · Hardware Software Security

September 2, 2026

Santiago Torres Arias

---

1 Installing the Required Tools

I use Arch Linux as my operating system. I use `pacman` to manage packages.

I found that the package names listed in the assignment correspond directly to packages in `pacman`. I used the `--needed` flag to avoid reinstalling packages if they were already present in my system.

Terminal

```
→ ricardo ~ sudo pacman -S --needed arm-none-eabi-gcc
→ ricardo ~ sudo pacman -S --needed arm-none-eabi-binutils
→ ricardo ~ sudo pacman -S --needed qemu-system-arm
```

Now, I ran the `-Ql` flag on each of them to see which files those packages brought to my computer. I realized that the output was super long, so I redirected it to a file.

Terminal

```
→ ricardo ~ pacman -Ql arm-none-eabi-gcc > arm-none-eabi-gcc-files.txt
→ ricardo ~ pacman -Ql arm-none-eabi-binutils > arm-none-eabi-binutils-files.txt
→ ricardo ~ pacman -Ql qemu-system-arm > qemu-system-arm-files.txt
```

For `arm-none-eabi-gcc`: It contains 2,969 entries. Using the folders, I grouped the contents like this:

1. The path `/usr/arm-none-eabi/include/c++/` contains ∼1,650 entries (∼56%) consisting mainly of C++ headers.
2. The path `/usr/arm-none-eabi/lib/` contains ∼250 entries (∼9%) related to the required libraries for the package.
3. The path `/usr/bin/` contains ∼20 entries (∼1%) related to the executables. I assume this because it is included in my `$PATH`.
4. The path `/usr/lib/gcc/arm-none-eabi/` contains ∼1,000 entries (35%) related to GCC drivers.
5. The last path `/usr/share/man/` contains manual pages for the installed commands.

For `arm-none-eabi-binutils`: It contains 84 entries.

1. The path `/usr/arm-none-eabi/bin/` contains ∼10 entries (∼12%) consisting mainly of binaries.
2. The path `/usr/arm-none-eabi/lib/` contains ∼20 entries (∼24%) related to the required libraries for the package.
3. The path `/usr/bin/` contains ∼15 entries (∼18%) related to the executables.
4. The path `/usr/share/man/` contains the remaining files (∼46%) for manual pages.

For `qemu-system-arm`: It contains only 16 entries.

1. The executable in the path `/usr/bin/`.
2. Some licenses in the path `/usr/share/licenses/`.
3. Manual pages at the path `/usr/share/man/`.

2 qemu-system-arm

Now, for this part of the assignment, I tried to run the command provided to run the "hello world," but I got this message.

Terminal

```
→ ricardo homework1 qemu-arm cpu-emu-test
zsh: command not found: qemu-arm
```

Checking with my trusty LLM, I realized that I was missing the package `qemu-user`, so I installed it using `pacman` as I did before.

Terminal

```
→ ricardo homework1 sudo pacman -S --needed qemu-user
```

For `qemu-user`: It contains only 45 entries.

1. Some required tool executables in the path `/usr/bin/`.
2. Manual pages at the path `/usr/share/man/` as usual.

Now, it runs correctly!

Terminal

```
→ ricardo homework1 qemu-arm cpu-emu-test
Hello world!
```

Continuing with the assignment, I ran the command to emulate a machine:

Terminal

```
→ ricardo   qemu-system-arm -machine xilinx-zynq-a9
VNC server running on ::1:5900
```

I was expecting a monitor interface as shown in the reading. I again asked my trusty LLM for options to solve this problem.

I have two options to address this:

1. To install `qemu-ui-gtk` that would provide my system with a GTK interface to the emulated machine.
2. To add the flags `-display none -monitor stdio` so it will directly open the monitor in my terminal emulator.

I chose option 2 because I prefer to avoid unnecessary packages. Now, my updated command works correctly:

Terminal

```
→ ricardo   qemu-system-arm -machine xilinx-zynq-a9 -display none -monitor stdio

QEMU 11.1.1 monitor - type 'help' for more information
(qemu)
```

Now the setup is completed!

3 Building ARM binaries

Let's save the file `babysteps.s` with the recommended formatting:

babysteps.s

```
1   .global _start
2
3   _start:
4   top:
5       eor r1, r1
6       b top
```

To build this, I ran the commands recommended in the readings:

Terminal

```
→ ricardo homework1 arm-none-eabi-as babysteps.s -o babysteps.o
→ ricardo homework1 arm-none-eabi-ld -g babysteps.o -o babysteps.elf
```

Investigating by myself, I was able to understand what those commands do:

- The first command `arm-none-eabi-as` builds an object file for ARM architecture. It reads the file `babysteps.s` and then the flag `-o` indicates the output path `babysteps.o`.
- The second command `arm-none-eabi-ld` links the object file assigning memory addresses and resolving symbols. It reads the file `babysteps.o` and then the flag `-o` indicates the output path `babysteps.elf`. The flag `-g` is just a compatibility flag; it is ignored.

Now, decomposing the assembly source code, I would describe it as follows:

- `.global _start` declares the symbol `_start` as global.
- `_start:` indicates that the symbol `_start` starts in that position.
- `top:` indicates that the symbol `top` starts in that position.
- `eor r1, r1` indicates an XOR operation. The evaluation of r1 $\oplus$ r1 is assigned to r1.
- `b top` indicates a jump to the `top:` symbol.

I would summarize the assembly as the following "equivalent" `C` code that is more human-readable:

babysteps.c

```c
int start(){
    int r1;
    while(1){
        r1 = r1 ^ r1;
    }
}
```

It contains an endless loop with an assignment of an XOR operation.